SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	V JAGADISH	**Reg. No.:**	192521352
Section / Batch:	B TECH IT	**Date:**	4/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Analyze Merge Sort execution by counting the number of comparisons and merge operations for different dataset sizes. Record results systematically. Interpret how these operations contribute to overall complexity. Justify why Merge Sort guarantees predictable performance

Code of Conduct

I JAGADISH.V(192521352)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JAGADISH.V

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Jagadish V
192521352RC

Q1
Merge Sort
100 marks

1/1 answered

Analyze Merge Sort execution by counting the number of comparisons and merge operations for different dataset sizes. Record results systematically. Interpret how these operations contribute to overall complexity. Justify why Merge Sort guarantees predictable performance.

Your Code

```
def merge_sort(arr):
    global comparisons,merges
    if len(arr)<=1:
        return arr
    mid=len(arr)
    left=merge_sort(arr[mid:])
    right=merge_sort(arr[mid:])
    merges+=1
    result=[]
    i=j=0
    while i<len(left)and j<len(right):
        comparisons+=1
        if left[i]<right[j]:
            result.append(left[i]);i+=1
        else:
            result.append(right[j]);j+=1
            return result +left[i:]+right[j:]
        arr=[38,27,43,3,9,82,10,14]
        sorted_arr=merge_sort(arr)
        print("sorted array:",sorted_arr)
        print("comparisions:",comparisions)
        print("merge operations:",merges)
```

Input

```
[38,27,43,3,9,82,10,14]
```

Output

```
Sorted Array: [3, 9, 10, 14, 27, 38, 43, 82]
Comparisons: 16
Merge Operations: 7
```